

IV BOB. NUMPY MASSIVLARIDA QIRQIB OLISH VA INDEKSLASH (SLICING & INDEXING IN NUMPY ARRAY)

Indekslash massivdagi ma'lum elementlarga ularning **pozitsiyasi (o'rni)** yoki **shartlar** asosida murojaat qilish uchun ishlatilsa, **slicing (kesib olish)** massivdan elementlarning kichik to'plamini ajratib olish uchun qo'llaniladi va u `massiv[start:stop:step]` sintaksisi orqali amalga oshiriladi. Bu mexanizm ham bir o'lchamli, ham ko'p o'lchamli massivlar uchun amal qiladi, bu esa kerakli qatorlar, ustunlar yoki ma'lumotlar diapazonini tanlab olishni osonlashtiradi.

4.1. Qirqib olish va indekslash (Slicing and Indexing)

NumPy massivini indekslash

Indekslash bir o'lchamli massivdan alohida elementlarni ajratib olish uchun ishlatiladi. Shuningdek, u ko'p o'lchamli massivlardan qatorlar, ustunlar yoki tekisliklarni (planes) ajratib olishda ham qo'llaniladi.

Tasavvur qiling, bizda quyidagi elementlardan tashkil topgan NumPy massivi bor: `arr = np.array([23, 21, 55, 65, 23])`

Ushbu massiv elementlariga ikki xil usulda — **musbat** (chapdan o'ngga) va **manfiy** (o'ngdan chapga, ya'ni teskari) indekslar orqali murojaat qilishimiz mumkin:

Indekslash jadvali:

Element (Qiymat)	23	21	55	65	23
Musbat indeks	0	1	2	3	4
Manfiy indeks	-5	-4	-3	-2	-1

Kodda qo'llanilishi:

```
import numpy as np

# Massivni yaratamiz
arr = np.array([23, 21, 55, 65, 23])

# 1. Musbat indeks orqali elementni olish
print(arr[0]) # Natija: 23 (birinchi element)
```

```
print(arr[3])    # Natija: 65 (to'rtinchi element)

# 2. Manfiy indeks orqali (oxiridan sanash)
print(arr[-1])  # Natija: 23 (eng oxirgi element)
print(arr[-3])  # Natija: 55 (oxiridan uchinchi element)
```

```
23
65
23
55
```

Eslatma: Musbat indekslash doim **0** dan boshlanishini unutmang. Manfiy indekslash esa massivning uzunligini bilmagan holatda, uning oxirgi elementlarini tezkor olish uchun juda qulaydir.

Indeks massivlari yordamida indekslash

Indeks massivlari bilan indekslash (Indexing with index arrays) NumPy massivining bir nechta elementlarini ularning indeks o'rinlari orqali bir vaqtning o'zida ajratib olish imkonini beradi. Slicing-dan farqli o'laroq, bu usul ma'lumotlarning **yangi nusxasini (copy)** qaytaradi.

Misol: Quyida biz kamayish tartibidagi massiv yaratamiz va undan aniq elementlarni ajratib olish uchun boshqa bir indekslar massividan foydalanamiz.

```
# 10 dan 1 gacha bo'lgan, -2 qadamli massiv yaratish
arr1 = np.arange(10, 1, -2)
print("Massiv:", arr1)

# Indeks massivi yordamida elementlarni olish
arr2 = arr1[[3, 1, 2]]
print("Berilgan indekslardagi elementlar:", arr2)
```

```
Massiv: [10  8  6  4  2]
Berilgan indekslardagi elementlar: [4 8 6]
```

Izoh: `arr1[[3, 1, 2]]` ifodasi original massivning 3, 1 va 2-indekslaridagi elementlarni tanlab olib, yangi massiv hosil qiladi. Natijada `[4, 8, 6]` massivi kelib chiqadi.

NumPy massivlarida indekslash turlari

NumPy-da ma'lumotlarga murojaat qilishning bir necha usullari mavjud. Quyida ularning asosiylari bilan tanishamiz:

1. Oddiy Slicing (Kesib olish) va Indekslash

Oddiy slicing va indekslash massivning muayyan elementlarini yoki elementlar oralig'ini ajratib olish uchun ishlatiladi. Bu usulning muhim xususiyati shundaki, u original massivning **nusxasini (copy)** emas, balki uning **ko'rinishini (view)** qaytaradi (ya'ni, xotirani tejaydi).

Umumiy sintaksis: `x[start:stop:step]`

- **start:** boshlang'ich indeks (bu indeksdagi element natijaga kiradi).
- **stop:** oxirgi indeks (bu indeksdagi element natijaga **kirmaydi**).
- **step:** elementlar orasidagi qadam (interval).

Misol:

```
# 0 dan 19 gacha bo'lgan sonlardan iborat massiv yaratish
a = np.arange(20)

print("a[15] =", a[15])           # 15-indeksdagi bitta element
print("a[-8:17:1] =", a[-8:17:1]) # Manfiy indeksdan boshlab
print("a[10:] =", a[10:])         # 10-indeksdan oxirigacha

a[15] = 15
a[-8:17:1] = [12 13 14 15 16]
a[10:] = [10 11 12 13 14 15 16 17 18 19]
```

Ellipsis (...) belgisidan foydalanish

Ko'p o'lchamli massivlar bilan ishlaganda, ba'zan biz massivning eng ichki qismidagi ma'lumotlarga kirishimiz kerak bo'ladi, lekin ungacha bo'lgan barcha o'lchamlarni bittalab yozib chiqish noqulaylik tug'diradi. Mana shunday hollarda **Ellipsis (...)** yordamga keladi.

Ellipsis — bu NumPy-dagi maxsus belgi bo'lib, u “qolgan barcha o'lchamlar” degan ma'noni anglatadi.

Misol: Quyidagi misolda bizda 3 o'lchamli (3D) massiv bor. Maqsadimiz: barcha qatlamlar va barcha qatorlardagi **ikkinchi elementni** (ya'ni 1-indeksdagi qiymatni) ajratib olish.

```
# 3 o'lchamli massiv yaratish
b = np.array( [ [1, 2, 3],
                [4, 5, 6],
                [7, 8, 9],
                [10, 11, 12]] )

# Ellipsis yordamida barcha o'lchamlardagi 1-indeksli elementlarni olish
print(b[..., 1])
```

```
[[ 2  5]
 [ 8 11]]
```

Bu qanday ishlaydi?

Odatda, 3D massivda (blok, qator, ustun) aniq bir ustunni olish uchun biz `b[:, :, 1]` deb yozishimiz kerak bo'lar edi. Bu yerda har bir ikki nuqta (:) bitta o'lchamni ifodalaydi.

- `b[..., 1]` yozuvi NumPy-ga shunday buyruq beradi: *"Massiv necha o'lchamli bo'lishidan qat'i nazar, oxirgi o'lchamga yetguncha bo'lgan hamma narsani qamrab ol va oxirida faqat 1-indeksdagi elementni tanla"*.

Nima uchun bu qulay?

1. **Kod qisqaligi:** Agar massiv 5 yoki 6 o'lchamli bo'lsa, `b[:, :, :, :, :, 1]` deb yozishdan ko'ra `b[..., 1]` deb yozish ancha oson.
2. **Moslashuvchanlik:** Kodingiz massivning o'lchamlari soni o'zgarganda ham (masalan, 3D dan 4D ga o'tsa) o'z-o'zidan ishlayveradi.

Qisqacha qoida: `...` belgisi massivning boshida, o'rtasida yoki oxirida kelishi mumkin va u har doim o'zi egallagan joydagi barcha o'lchamlarni to'liq tanlab oladi.

2. Kengaytirilgan Indeksflash (Advanced Indexing)

NumPy-da kengaytirilgan indeksflash butun sonlar (integer) yoki mantiqiy (boolean) massivlar yordamida murakkab ma'lumotlar tuzilmalarini ajratib olish imkonini beradi.

Muhim farqi: Oddiy slicing-dan farqli o'laroq, kengaytirilgan indeksflash har doim ma'lumotlarning **nusxasini (copy)** qaytaradi (original massivning "ko'rinishini" emas).

Kengaytirilgan indeksflash quyidagi hollarda sodir bo'ladi:

- Indeks obykti butun sonli yoki mantiqiy `ndarray` bo'lganda;
- Indeks kamida bitta ketma-ketlik obyektiga ega bo'lgan tuple (kortej) bo'lganda;
- Indeks tuple bo'lmagan ketma-ketlik obykti bo'lganda.

Kengaytirilgan indekslashning ikki turi mavjud:

1. Butun sonli massiv yordamida indekslash (Purely integer indexing)
2. Mantiqiy (Boolean) indekslash

Butun sonli massiv yordamida indekslash

Bu usul elementlarni butun sonli massivlar yordamida tanlaydi. Har bir o'lchamdan olingan indekslar juftligi aniq bir elementning manzilini belgilaydi.

Misol:

```
a = np.array( [[1, 2], [3, 4], [5, 6]] )

# (0,0), (1,0) va (2,1) pozitsiyadagi elementlarni ajratib olish
print( a[[0, 1, 2], [0, 0, 1]] )
```

```
[1 3 6]
```

Izoh: Bu yerda birinchi massiv `[0, 1, 2]` qator indekslarini, ikkinchi massiv `[0, 0, 1]` esa mos ravishda ustun indekslarini bildiradi. Natijada `a[0,0]`, `a[1,0]` va `a[2,1]` elementlari tanlab olinadi.

Mantiqiy (Boolean) indekslash

Ushbu usulda indeks sifatida mantiqiy ifodalar (shartlar) ishlatiladi. Massivning faqat ushbu shartni qanoatlantiradigan (ya'ni `True` bo'lgan) elementlari qaytariladi. Bu usul ma'lumotlarni filtrlash uchun juda qulay.

1-misol: 50 dan katta sonlarni tanlash

```
a = np.array([10, 40, 80, 50, 100])

# Shart: a ichidagi element 50 dan katta bo'lsin
print(a[a > 50])
```

```
[ 80 100]
```

2-misol: Qatorlar yig'indisi 10 ga qoldiqsiz bo'linadigan qatorlarni tanlash

```
b = np.array([[5, 5], [4, 5], [16, 4]])

# Har bir qator yig'indisini hisoblaymiz (axis=-1)
res = b.sum(-1)

# Faqat yig'indisi 10 ga bo'linadigan qatorlarni chiqaramiz
print(b[res % 10 == 0])
```

```
[[ 5  5]
 [16  4]]
```

Izoh: `res` massivi `[10, 9, 20]` qiymatlarini oladi. `res % 10 == 0` sharti esa `[True, False, True]` mantiqiy massivini hosil qiladi. Shuning uchun faqat 1 va 3-qatorlar natijaga chiqadi.

4.2. Massiv indekslash bo'yicha amaliy misollar (More Examples of Array Indexing)

NumPy Massivlarini Indekslish (Indexing)

NumPy-da massivlarni indekslash — bu massiv ichidagi aniq elementlarga yoki ma'lumotlar guruhiga ularning manzili (pozitsiyasi) orqali murojaat qilish usulidir. Bu funksiya yirik ma'lumotlar to'plami (dataset) bilan ishlashda kerakli ma'lumotlarni olish, o'zgartirish va boshqarishni ancha osonlashtiradi.

1. 1D (Bir o'lchamli) massivlarda elementlarga murojaat qilish

Bir o'lchamli NumPy massivi — bu tartiblangan qiymatlar ketma-ketligi bo'lib, har bir qiymat o'zining **indeksiga** ega.

Asosiy qoidalar:

- **Indekslish 0 dan boshlanadi:** Massivning birinchi elementi `0` indeksida, ikkinchisi `1` indeksida va hokazo joylashadi.
- **Kvadrat qavslar:** Elementga murojaat qilish uchun massiv nomidan keyin kvadrat qavs ichida indeks ko'rsatiladi:
`arr[indeks]`.
- **Manfiy indekslash:** Indekslar massivning oxiridan ham sanalishi mumkin. Bunda `-1` eng oxirgi elementni, `-2` esa oxiridan bittadan oldingi elementni bildiradi.

Misol:

```
# 1D massiv yaratish
arr = np.array([10, 20, 30, 40, 50])

# Birinchi elementga murojaat (0-indeks)
print(arr[0])

# Oxirgi elementga murojaat (manfiy indeks)
print(arr[-1])
```

10

50

Bu usul orqali siz massiv ichidagi istalgan nuqtadagi ma'lumotni soniyalarda ajratib olishingiz va agar kerak bo'lsa, uni yangi qiymatga o'zgartirishingiz mumkin (masalan, `arr[0] = 100`).

2. Ko'p o'lchamli massivlarda elementlarga murojaat qilish

Ushbu bo'limda biz 2D va 3D massivlardagi elementlarga aniq indekslar yordamida qanday murojaat qilishni ko'rib chiqamiz.

2D massivlar: Elementlarga qator va ustun indekslarini `matrix[qator, ustun]` ko'rinishida ko'rsatish orqali murojaat qilishimiz mumkin.

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
  
print(matrix[1, 2])
```

6

Bu yerda `matrix[1, 2]` ikkinchi qator (indeks 1) va uchinchi ustundagi (indeks 2) elementni, ya'ni **6** ni ajratib oladi.

3D massivlar: Buni 2D massivlar to'plami (steck) sifatida tasavvur qilish mumkin. Bizga uchta indeks kerak bo'ladi:

- **Chuqurlik (Depth):** 2D qatlamni (kesimni) belgilaydi.
- **Qator (Row):** Shu qatlam ichidagi qatorni belgilaydi.
- **Ustun (Column):** Shu qator ichidagi ustunni belgilaydi.

Elementlarga `matrix[chuqurlik, qator, ustun]` ko'rinishida murojaat qilinadi.

```
cube = np.array([[[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]],  
                 [[10, 11, 12],  
                  [13, 14, 15],  
                  [16, 17, 18]]])  
  
print(cube[1, 2, 0])
```

16

3. Massivlarni kesib olish (Slicing)

Bu usul bizga `start:stop:step` formatidan foydalangan holda elementlar diapazonini ajratib olish imkonini beradi. Slicing ham 1D, ham ko'p o'lchamli massivlar uchun qo'llanilishi mumkin, bu esa elementlar diapazoni yoki kichik matritsalarini osongina tanlash imkonini beradi.

1D massivlarni kesib olish: 1D massiv uchun slicing `start` va `stop` indeksleri orasidagi elementlar to'plamini qaytaradi.

```
arr = np.array([0, 1, 2, 3, 4, 5])
```

```
print(arr[1:4])
```

```
[1 2 3]
```

Bu yerda `arr[1:4]` massivni 1-indeksdan boshlab 4-indeksgacha (4-indeks kirmaydi) kesib oladi, shuning uchun natija `[1, 2, 3]` bo'ladi.

Ko'p o'lchamli massivlarni kesib olish: Bunda slicing har bir o'lchamga alohida qo'llanilishi mumkin, bu esa ma'lumotlarning kichik bloklarini yoki submatritsalarini ajratib olish imkonini beradi.

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print(matrix[0:2, 1:3])
```

```
[[2 3]
```

```
[5 6]]
```

Izoh:

- `0:2` — birinchi va ikkinchi qatorlarni tanlaydi (0 va 1-indeks).
- `1:3` — ikkinchi va uchinchi ustunlarni tanlaydi (1 va 2-indeks).

Natijada ushbu qator va ustunlar kesishmasidagi kichik matritsa hosil bo'ladi.

4. Mantiqiy (Boolean) Indekslish

Ushbu usul massiv elementlarini ma'lum bir **shart** asosida filtrlash imkonini beradi va faqat shu shartga mos keladigan elementlarni qaytaradi. Biz shart yordamida mantiqiy massiv (Boolean array) yaratamiz va undan elementlarni tanlashda foydalanamiz. Shuningdek, shartlarni mantiqiy operatorlar orqali birlashtirish ham mumkin.

```
arr = np.array([10, 15, 20, 25, 30])
```



```
# 20 dan katta elementlarni filtrlash
print(arr[arr > 20])
```

```
[25 30]
```

Izoh: `arr > 20` sharti 20 dan katta elementlar uchun `True` qiymatini qaytaradi, shuning uchun faqat 25 va 30 tanlab olinadi va ekranga chiqariladi.

Shuningdek, shartlarni birlashtirish uchun **& (VA/AND)**, **| (YOKI/OR)** va **~ (EMAS/NOT)** kabi mantiqiy operatorlardan foydalanish mumkin.

```
arr = np.array([10, 15, 20, 25, 30])

# 10 dan katta VA 30 dan kichik bo'lgan elementlarni tanlash
print(arr[(arr > 10) & (arr < 30)])
```

```
[15 20 25]
```

Izoh: Birlashtirilgan shart 10 dan katta va 30 dan kichik bo'lgan elementlarni tanlaydi, natijada `[15, 20, 25]` massivi hosil bo'ladi.

5. Fancy Indexing (Murakkab indekslash)

Fancy Indexing (shuningdek, kengaytirilgan indekslash deb ham ataladi) massiv elementlariga boshqa bir massiv yoki indekslar ro'yxati yordamida murojaat qilish imkonini beradi. Bu usul bir vaqtning o'zida bir nechta elementni, hatto ular yonma-yon joylashmagan bo'lsa ham, tanlab olishga xizmat qiladi. Bu massivning turli nuqtalaridan aniq qiymatlarni terib olishni osonlashtiradi.

```
arr = np.array([10, 20, 30, 40, 50])
indices = [0, 2, 4]

print(arr[indices])
```

```
[10 30 50]
```

Izoh: Dastur `[0, 2, 4]` ro'yxatidan foydalanib, massivning aynan shu pozitsiyalaridagi elementlarni tanlab oladi va `[10, 30, 50]` natijasini qaytaradi.

6. Butun sonli massiv yordamida indekslash (Integer Array Indexing)

Bu usul **Fancy Indexing**ga juda o'xshash bo'lib, boshqa bir massivdan bir nechta elementni tanlash uchun butun sonli massivdan foydalanadi. Ushbu metod bir-biriga tutash bo'lmagan o'rinlardagi elementlarga murojaat qilish

imkonini bergani uchun tarqoq ma'lumotlar nuqtalarini ajratib olishda juda foydalidir.

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr[[0, 2, 4]])
```

```
[1 3 5]
```

Izoh: `[0, 2, 4]` butun sonli massivi ushbu indekslardagi elementlarni tanlaydi va `[1, 3, 5]` natijasini qaytaradi.

7. Indeksplashda Ellipsis (...) belgisidan foydalanish

Ellipsis (...) belgisi aniq ko'rsatilmagan barcha o'lchamlarni tanlash uchun ishlatiladi. Bu ko'p o'lchamli massivlarda har bir o'lchamni bittalab yozib chiqishni istamasak, juda qo'l keladi.

```
# 4x4x4 o'lchamli tasodifiy sonlardan iborat kub yaratish
```

```
cube = np.random.rand(4, 4, 4)
```

```
print(cube[..., 0])
```

```
[[0.33701682 0.36288464 0.78606831 0.04304207]
 [0.62084339 0.88124178 0.12820329 0.73282342]
 [0.85138939 0.55888385 0.19372724 0.84095461]
 [0.62540634 0.81230526 0.62856344 0.89555438]]
```

Izoh: Bu yerda `...` dastlabki ikki o'lchamdagi barcha elementlarni tanlaydi, `0` esa oxirgi o'lcham bo'yicha har bir blokda birinchi elementni ajratib oladi. *Eslatma:* `np.random.rand` ishlatilgani uchun natijada tasodifiy sonlar hosil bo'ladi.

8. `np.newaxis` yordamida yangi o'lcham qo'shish

`np.newaxis` kalit so'zi massivga yangi o'q (o'lcham) qo'shadi. Bu 1D massivni qator yoki ustun vektoriga aylantirishda yordam beradi.

```
arr = np.array([1, 2, 3])
```

```
# 1D massivni ustun vektoriga aylantirish
```

```
print(arr[:, np.newaxis])
```

```
[[1]
 [2]
 [3]]
```

Izoh: Bu yerda yangi o'q qo'shilishi natijasida 1D massiv (3, 1) shaklidagi 2D ustun vektoriga aylanadi.

9. Massiv elementlarini o'zgartirish

Biz massiv elementlarini **indekslash** yoki **slicing** yordamida to'g'ridan-to'g'ri o'zgartirishimiz mumkin. Bu massivdagi aniq bir elementni yoki elementlar oralig'ini yangilashni osonlashtiradi.

```
arr = np.array([1, 2, 3, 4])

# 1 va 2-indeksdagi elementlarni 99 ga o'zgartirish
arr[1:3] = 99

print(arr)

[ 1 99 99  4]
```

Izoh: `arr[1:3]` slice qismi 1 va 2-indeksdagi elementlarni tanlaydi va ularni 99 qiymati bilan almashtiradi. Ushbu usullarni o'zlashtirish NumPy yordamida ma'lumotlarni yanada samarali tahlil qilish va boshqarish imkonini beradi.

4.3. Ko'p o'lchovli massivlarda satrlarga murojaat qilish uchun qirqib olish (Slicing Multidimensional Array for accessing different rows)

Ko'p o'lchamli NumPy massivining turli qatorlariga murojaat qilish

Ko'p o'lchamli NumPy massivida birinchi qator, oxirgi ikki qator yoki o'rtadagi qatorlar kabi turli qismlarga murojaat qilish kerak bo'lganda, buni **slicing** (kesib olish) yoki **fancy indexing** (murakkab indekslash) yordamida amalga oshirish mumkin. NumPy berilgan shartlar asosida aniq qatorlarni tanlashning oddiy usullarini taqdim etadi.

2D massivda (Matritsada)

1-misol: 2D massivning birinchi va oxirgi qatoriga murojaat qilish

Ushbu misolda biz fancy indexing usulidan foydalanib, massivning 0-indeksdagi (birinchi) va 2-indeksdagi (uchinchi/oxirgi) qatorlarini bir vaqtda ajratib olamiz.

```
# 3x3 o'lchamli 2D massiv yaratish
arr = np.array([[10, 20, 30],
                [40, 5, 66],
```

```

        [70, 88, 94]])

print("Asl massiv:")
print(arr)

# 0 va 2-indeksdagi qatorlarni tanlash
res = arr[[0, 2]]

print("\nTanlab olingan qatorlar:")
print(res)

```

```

Asl massiv:
[[10 20 30]
 [40  5 66]
 [70 88 94]]

```

```

Tanlab olingan qatorlar:
[[10 20 30]
 [70 88 94]]

```

Izoh:

- `arr[[0, 2]]` yozuvi NumPy-ga indekslar ro'yxatini uzatadi.
- Birinchi indeks `0` bo'lgani uchun birinchi qator (`[10, 20, 30]`) olinadi.
- Ikkinchi indeks `2` bo'lgani uchun uchinchi qator (`[70, 88, 94]`) olinadi.
- Natijada ushbu ikki qatordan iborat yangi massiv hosil bo'ladi.

Bu usul qatorlar yonma-yon bo'lmagan holatlarda juda qo'l keladi. Agar sizga faqat diapazon (masalan, birinchidan ikkinchigacha) kerak bo'lsa, `arr[0:2]` ko'rinishidagi oddiy slicing-dan foydalangan ma'qul.

2-misol: 2D NumPy massivining o'rta qatoriga murojaat qilish

Ushbu misolda biz 2D massivning (matritsaning) aynan o'rtasida joylashgan qatorni ajratib olishni ko'rib chiqamiz. Buning uchun massiv nomidan keyin kvadrat qavs ichida kerakli qatorning indeksini ko'rsatish kifoya.

```

# 3x4 o'lchamli 2D massiv yaratish
arr = np.array([[101, 20, 3, 10],
                 [40, 5, 66, 7],
                 [70, 88, 9, 141]])

print("Asl massiv:")

```

```
print(arr)

# 1-indeksdagi (o'rtadagi) qatorni ajratib olish
res = arr[1]

print("\nTanlab olingan qator:")
print(res)
```

Asl massiv:

```
[[101  20   3  10]
 [ 40   5  66   7]
 [ 70  88   9 141]]
```

Tanlab olingan qator:

```
[40  5 66  7]
```

Izoh:

- NumPy-da indekslash **0** dan boshlanadi.
- Bizda 3 ta qator bor: 0-indeks (birinchi), 1-indeks (ikkinchi/o'rta) va 2-indeks (uchinchi).
- `arr[1]` buyrug'i massivning ikkinchi qatorini to'liqligicha ajratib oladi.

Agar massiv juda katta bo'lsa va siz o'rta qatorning indeksini aniq bilmasangiz, uni quyidagicha dinamik hisoblashingiz ham mumkin:

```
res = arr[len(arr) // 2].
```

3-misol: 2D NumPy massivining aniq qator va ustunlariga murojaat qilish

Ushbu misolda biz **slicing** (kesib olish) usuli yordamida matritsaning ma'lum bir qismini (kichik submatoritsani) ajratib olishni ko'rib chiqamiz. Bunda bir vaqtning o'zida ham qatorlar, ham ustunlar bo'yicha diapazon belgilanadi.

```
# 3x3 o'lchamli 2D massiv yaratish
arr = np.array([[12, 15, 18],
                [25, 30, 35],
                [40, 45, 50]])

print("Asl massiv:")
print(arr)

# Birinchi 2 ta qator va birinchi 2 ta ustunni ajratib olish
res = arr[:2, :2]
```

```
print("\nTanlab olingan elementlar (submatritsa):")
print(res)
```

Asl massiv:

```
[[12 15 18]
 [25 30 35]
 [40 45 50]]
```

Tanlab olingan elementlar (submatritsa):

```
[[12 15]
 [25 30]]
```

Izoh:

Slicing operatsiyasida verguldan oldingi qism qatorlarni, verguldan keyingi qism esa ustunlarni bildiradi: `arr[qatorlar, ustunlar]`.

1. **Qatorlar kesimi (:2)**: Bu 0-indeksdan boshlab 2-indeksgacha bo'lgan qatorlarni tanlaydi (lekin 2-indeks kirmaydi). Ya'ni, 0 va 1-qatorlar tanlanadi.
2. **Ustunlar kesimi (:2)**: Bu ham 0-indeksdan boshlab 2-indeksgacha bo'lgan ustunlarni tanlaydi (2-indeks kirmaydi). Ya'ni, 0 va 1-ustunlar tanlanadi.
3. **Kesiishma**: Tanlangan qatorlar va ustunlar kesishmasida hosil bo'lgan 2×2 o'lchamli yangi massiv natija sifatida qaytariladi.

Bu usul katta hajmli jadvallardan (masalan, datasetlardan) ma'lum bir blokni ajratib olish va tahlil qilish uchun juda qulaydir.

3D massivlarda qatorlarga murojaat qilish

3D massivlarda ma'lumotlar bir nechta qatlamlardan (bloklardan) iborat bo'ladi. Slicing yordamida biz har bir qatlamning ma'lum bir qismini, masalan, o'rtadagi qatorlarini ajratib olishimiz mumkin.

1-misol: 3D NumPy massivining o'rta qatorlariga murojaat qilish

Ushbu misolda har bir qatlamdagi (blokdagi) o'rta qatorni (1 -indeksdagi qator) qanday qilib bir vaqtning o'zida ajratib olish ko'rsatilgan.

```
# 2x3x3 o'lchamli 3D massiv yaratish (2 ta qatlam, har biri 3x3)
arr = np.array([[[10, 25, 70], [30, 45, 55], [20, 45, 7]],
                [[50, 65, 8], [70, 85, 10], [11, 22, 33]]])

print("Asl massiv:")
print(arr)
```

```
# Har bir qatlamdan 1-indeksdagi (o'rta) qatorni ajratib olish
res = arr[:, [1]]

print("\nTanlab olingan qatorlar:")
print(res)
```

Asl massiv:

```
[[[10 25 70]
   [30 45 55]
   [20 45 7]]

 [[50 65 8]
  [70 85 10]
  [11 22 33]]]
```

Tanlab olingan qatorlar:

```
[[[30 45 55]]

 [[70 85 10]]]
```

Izoh:

3D massivda indekslash `arr[qatlam, qator, ustun]` tartibida bo'ladi:

1. **Birinchi o'lcham (:)**: Ikki nuqta barcha qatlamlarni (bizda 2 ta qatlam bor) tanlashni bildiradi.
2. **Ikkinchi o'lcham ([1])**: Har bir qatlamning aynan 1-indeksdagi (o'rtadagi) qatorini tanlaydi.
3. **Uchinchi o'lcham (bo'sh)**: Biz ustunlarni ko'rsatmaganimiz uchun, tanlangan qatordagi barcha ustunlar avtomatik ravishda olinadi.

Natijada:

- 1-qatlamdan: `[30, 45, 55]`
- 2-qatlamdan: `[70, 85, 10]` qatorlari ajratib olinib, yangi 3D massiv ko'rinishida taqdim etiladi.

2-misol: 3D NumPy massivining birinchi va oxirgi qatorlariga murojaat qilish

Ushbu misolda biz 3D massivning har bir qatlamidan (blokidan) bir vaqtning o'zida ham birinchi (0-indeks), ham oxirgi (2-indeks) qatorlarni ajratib olishni ko'rib chiqamiz.

```
# 3x3x3 o'lchamli 3D massiv yaratish (3 ta qatlam, har biri 3x3 matritsa)
arr = np.array([[10, 25, 70], [30, 45, 55], [20, 45, 7]],
               [[50, 65, 8], [70, 85, 10], [11, 22, 33]],
               [[19, 69, 36], [1, 5, 24], [4, 20, 96]])

print("Asl massiv:")
print(arr)

# Har bir qatlamdan 0 va 2-indeksdagi qatorlarni ajratib olish
res = arr[:, [0, 2]]

print("\nTanlab olingan qatorlar:")
print(res)
```

Asl massiv:

```
[[[10 25 70]
   [30 45 55]
   [20 45  7]]

 [[50 65  8]
   [70 85 10]
   [11 22 33]]

 [[19 69 36]
   [ 1  5 24]
   [ 4 20 96]]]
```

Tanlab olingan qatorlar:

```
[[[10 25 70]
   [20 45  7]]

 [[50 65  8]
   [11 22 33]]

 [[19 69 36]
   [ 4 20 96]]]
```

Izoh:

3D massivda `arr[qatlam, qator, ustun]` tartibi saqlanib qoladi:

1. **:** (**Barcha qatlamlar**): Birinchi o'lchamda **:** belgisi ishlatilgani uchun barcha 3 ta qatlam (blok) qamrab olinadi.
2. **[0, 2]** (**Tanlangan qatorlar**): Ikkinchi o'lchamda indekslar ro'yxati berilgan. Bu NumPy-ga har bir qatlamning **birinchi (0)** va **uchinchi (2)** qatorlarini olishni buyuradi.

3. **Ustunlar:** Uchinchi o'lcham ko'rsatilmagani uchun ushbu qatorlardagi barcha ustunlar (qiymatlar) to'liq olinadi.

Natijada har bir blok o'zining o'rta qatorini yo'qotadi va faqat "chekka" qatorlaridan tashkil topgan yangi 3D massiv hosil bo'ladi.

3-misol: 3D NumPy massivining aniq qator va ustunlariga murojaat qilish

Ushbu misolda har bir qatlam (2D qism) uchun bir vaqtning o'zida ham qatorlar, ham ustunlar bo'yicha kesib olish (slicing) qanday bajarilishini ko'ramiz. Bu usul har bir blokdan kichikroq sub-matritsalarini ajratib olish imkonini beradi.

```
# 3x3x3 o'lchamli 3D massiv yaratish
arr = np.array([
    [[5, 10, 15], [20, 25, 30], [35, 40, 45]],
    [[2, 4, 6], [8, 10, 12], [14, 16, 18]],
    [[7, 14, 21], [28, 35, 42], [49, 56, 63]]
])

print("Asl massiv:")
print(arr)

# Har bir 2D qatlamdan birinchi 2 ta qator va birinchi 2 ta ustunni ajratish
res = arr[:, :2, :2]

print("\nTanlab olingan elementlar:")
print(res)
```

Asl massiv:

```
[[[ 5 10 15]
   [20 25 30]
   [35 40 45]]
```

```
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]
```

```
[[ 7 14 21]
 [28 35 42]
 [49 56 63]]]
```

Tanlab olingan elementlar:

```
[[[ 5 10]
   [20 25]]
```

```
[[ 2  4]
 [ 8 10]]
```

```
[[ 7 14]
 [28 35]]]
```

Izoh:

Slicing uchta o'lcham bo'yicha amalga oshirilmoqda:

```
arr[qatlam, qator, ustun]
```

1. **:** (**Barcha qatlamlar**): Birinchi o'lchamdagi ikki nuqta barcha uchta qatlamni tanlashni bildiradi.
2. **:2** (**Qatorlar kesimi**): Ikkinchi o'lchamda birinchi ikkita qator tanlanadi (0 va 1-indekslar).
3. **:2** (**Ustunlar kesimi**): Uchinchi o'lchamda birinchi ikkita ustun tanlanadi (0 va 1-indekslar).

Natijada: Har bir qatlamdan 3×3 matritsaning yuqori chap burchagidagi 2×2 o'lchamli qismi qirqib olinadi.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy massivlarida indekslash va qirqib olish (**slicing**) tushunchalarining asosiy farqi nimada?
2. Bir o'lchamli massiv elementlariga musbat va manfiy indekslar orqali murojaat qilish qoidalarini tushuntirib bering.

3. Oddiy `slicing` amali original massivning nusxasini qaytaradimi yoki ko'rinishini (`view`)?
4. `arr[start:stop:step]` sintaksisidagi har bir parametr nima vazifani bajarishini izohlang.

2-daraja: Funksiyalarning ishlash mexanizmi

5. Ko'p o'lchamli massivlar bilan ishlashda ellipsis (`...`) belgisi qanday qulaylik yaratadi?
6. Kengaytirilgan indekslash (`advanced indexing`) sodir bo'ladigan holatlarni sanab o'ting va uning xotira bilan ishlash xususiyatini ayting.
7. `np.newaxis` kalit so'zi massivga qanday ta'sir qiladi va u ko'pincha nima maqsadlarda ishlatiladi?
8. Butun sonli massiv yordamida indekslash (`integer array indexing`) qanday holatlarda foydali hisoblanadi?

3-daraja: Amaliy tahlil va murakkab indekslash

9. Mantiqiy (`boolean`) indekslash orqali ma'lumotlarni filtrlash qanday amalga oshiriladi? Bir nechta shartlarni birlashtirish uchun qaysi operatorlardan foydalaniladi?
10. `Fancy indexing` tushunchasini 1D massiv misolida tushuntirib bering.
11. 3D massivlarda qatlam, qator va ustunlar bo'yicha kesib olish (`arr[qatlam, qator, ustun]`) qanday bajariladi?
12. Massiv elementlarini `slicing` yordamida guruhli tarzda o'zgartirish jarayonining mohiyatini tushuntiring.